



2-6-2026

# **INFORME PRACTICA 4**

SISTEMAS OPERATIVOS



Marcos Encinas Lozano

## INDICE

|                                              |   |
|----------------------------------------------|---|
| <b>GitHub</b> .....                          | 2 |
| <b>INTRODUCCIÓN</b> .....                    | 2 |
| <b>OBJETIVOS</b> .....                       | 2 |
| <b>EXPLICACIÓN DEL PROGRAMA C++</b> .....    | 3 |
| 1. Lectura del archivo JSON.....             | 3 |
| 2. Validación de matrices 16×16 .....        | 3 |
| 3. Interpolación bilineal a 128×128 .....    | 3 |
| 4. Construcción del payload JSON .....       | 3 |
| 5. Envío HTTP mediante libcurl .....         | 4 |
| <b>EXPLICACIÓN DEL SERVIDOR PYTHON</b> ..... | 4 |
| 1. Recepción del JSON .....                  | 4 |
| 2. Reconstrucción de la matriz.....          | 4 |
| 3. Visualización con Matplotlib.....         | 4 |
| 4. Guardado automático de imágenes .....     | 4 |
| <b>CAPTURAS DE PANTALLA</b> .....            | 5 |
| <b>CONCLUSIONES</b> .....                    | 7 |
| <b>INCIDENCIAS</b> .....                     | 8 |

## GitHub

<https://github.com/marcosEncloz/Practica-4-Marcos-Encinas-Lozano-SO>

## INTRODUCCIÓN

En esta práctica se ha desarrollado un sistema completo de procesamiento y visualización de datos táctiles utilizando C++ y Python. El cliente, implementado en C++, es capaz de:

- Leer un archivo JSON con 50 capturas táctiles.
- Validar cada matriz  $16 \times 16$ .
- Interpolarla a  $128 \times 128$  mediante interpolación bilineal.
- Enviar cada captura al servidor mediante una petición HTTP POST.

El servidor, implementado en Python con Flask, recibe las matrices, las visualiza utilizando Matplotlib y genera automáticamente las imágenes:

capture\_0.png

capture\_1.png

...

capture\_49.png

Esta práctica permite afianzar conocimientos sobre JSON, modularización en C++, comunicación HTTP, uso de librerías externas (libcurl), servidores web en Python y visualización científica.

## OBJETIVOS

El objetivo principal de la práctica es implementar un sistema capaz de:

- Leer un archivo JSON con múltiples capturas.
- Validar matrices de entrada.
- Realizar interpolación bilineal  $16 \times 16 \rightarrow 128 \times 128$ .
- Construir un payload JSON para cada captura.
- Enviar los datos mediante HTTP POST usando libcurl.
- Implementar un servidor Flask que reciba los datos.
- Visualizar las matrices con Matplotlib.
- Guardar automáticamente las imágenes generadas.

El objetivo educativo es reforzar el uso de JSON, modularización en C++, comunicación cliente-servidor, visualización en Python y automatización del flujo de datos.

## EXPLICACIÓN DEL PROGRAMA C++

### 1. Lectura del archivo JSON

El programa comienza leyendo: tactile\_captures\_50.json

Mediante la función:

```
auto jsonData = readJSON("tactile_captures_50.json");
```

Se extraen las 50 capturas contenidas en el archivo.

### 2. Validación de matrices 16×16

Cada captura contiene una matriz 16×16. Antes de procesarla, se valida:

```
if (!validateMatrix(m)) { ... }
```

Esto asegura que:

- Tiene tamaño correcto.
- No contiene valores inválidos.
- No hay datos corruptos.

### 3. Interpolación bilineal a 128×128

La práctica exige aumentar la resolución de la matriz. Para ello se implementa:

```
Matrix128 out = interpolateBilinear(m);
```

La interpolación bilineal genera una matriz suave y continua, adecuada para la visualización térmica.

### 4. Construcción del payload JSON

Cada captura se empaqueta en un JSON:

```
payload["capture_id"] = i;
```

```
payload["width"] = 128;
```

```
payload["height"] = 128;
```

```
payload["data"] = out;
```

Este JSON será enviado al servidor Python.

## 5. Envío HTTP mediante libcurl

El envío se realiza con:

```
sendMatrixToServer(payload, "http://127.0.0.1:5000/upload");
```

Si el servidor responde correctamente:

```
{"status":"ok","saved":"images/capture_i.png"}
```

El cliente muestra:

OK

# EXPLICACIÓN DEL SERVIDOR PYTHON

## 1. Recepción del JSON

El servidor Flask expone el endpoint:

```
/upload
```

Recibe el JSON enviado por el cliente:

```
data = request.get_json()
```

## 2. Reconstrucción de la matriz

La matriz interpolada se convierte a NumPy:

```
matrix = np.array(data["data"])
```

Esto permite visualizarla fácilmente.

## 3. Visualización con Matplotlib

El enunciado exige:

- `plt.imshow(matrix, cmap="inferno")`
- `plt.colorbar(label="Presión")`
- `plt.title("Mapa táctil")`

El servidor genera exactamente esta visualización.

## 4. Guardado automático de imágenes

Cada captura se guarda como:

images/capture\_0.png

images/capture\_1.png

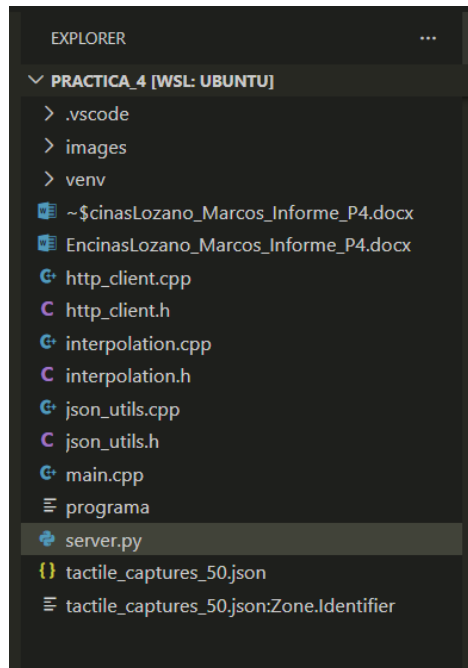
...

## Practica 4 - SO

images/capture\_49.png

Cumpliendo el requisito de la práctica.

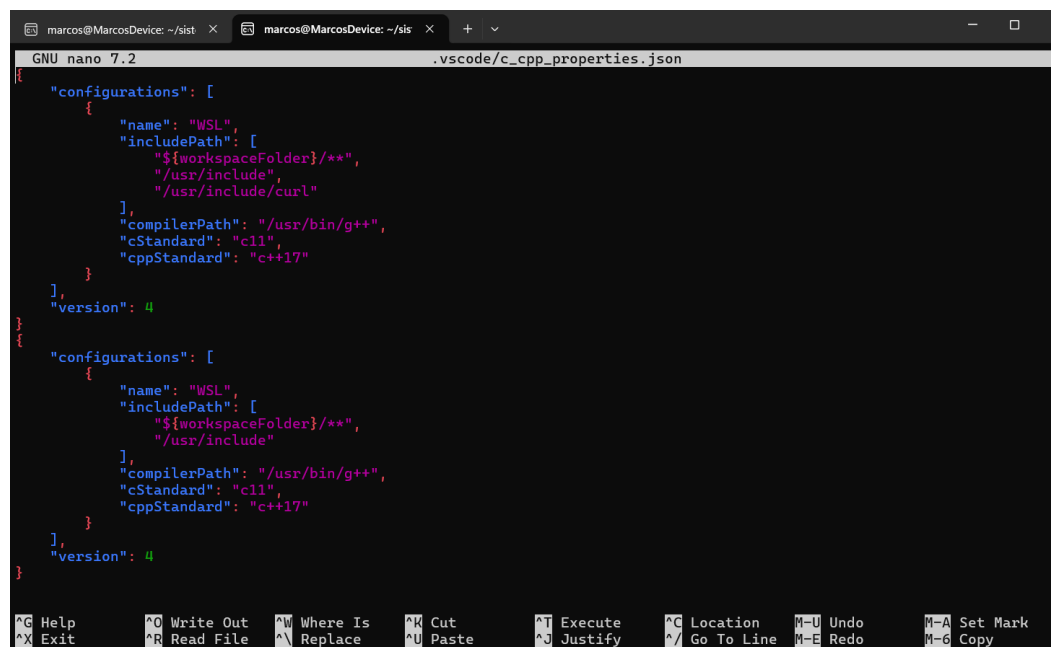
## CAPTURAS DE PANTALLA



*imagen de los diferentes archivos necesarios para la realización de la práctica.*

| Port | Forwarded Address | Running Process           | Origin         |
|------|-------------------|---------------------------|----------------|
| 5000 | 127.0.0.1:5000    | python3 server.py (21528) | Auto Forwarded |

### Información del servidor Python



## Practica 4 - SO

```
{
  "configurations": [
    {
      "name": "WSL",
      "includePath": [
        "${workspaceFolder}/**",
        "/usr/include",
        "/usr/include/curl"
      ],
      "compilerPath": "/usr/bin/g++",
      "cStandard": "c11",
      "cppStandard": "c++17"
    }
  ],
  "version": 4
}
{
  "configurations": [
    {
      "name": "WSL",
      "includePath": [
        "${workspaceFolder}/**",
        "/usr/include"
      ],
      "compilerPath": "/usr/bin/g++",
      "cStandard": "c11",
      "cppStandard": "c++17"
    }
  ],
  "version": 4
}
```

*Código .vscode/c\_cpp\_properties.json*

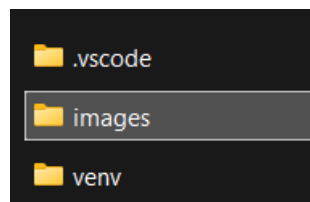
```
(venv) marcos@MarcosDevice:~/sistemas_operativos/practica_4$ CTRL + C
python3 server.py
CTRL: command not found
* Serving Flask app 'server'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [24/May/2026 21:23:17] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:18] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:18] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:18] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:18] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:18] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:18] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:18] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:18] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:18] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:19] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:19] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:19] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:19] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:19] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:19] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:19] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [24/May/2026 21:23:19] "POST /upload HTTP/1.1" 200 -
```

*Funcionamiento servidor Python*

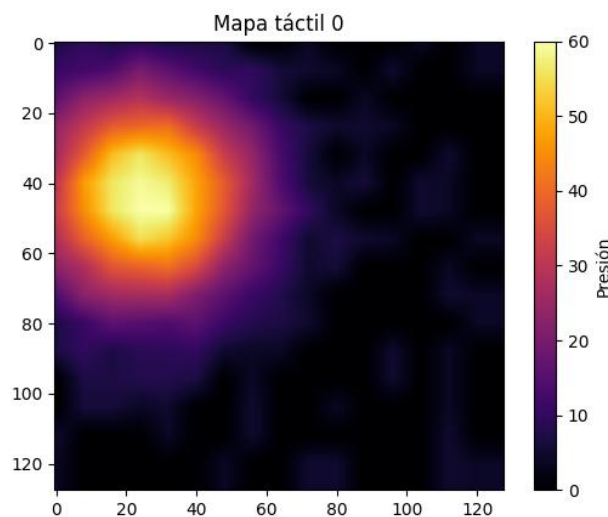
## Practica 4 - SO

```
marcos@MarcosDevice:~/sistemas_operativos/practica_4$ ./programa
Enviando captura 0...
{"saved":"images/capture_0.png","status":"ok"}
OK
Enviando captura 1...
{"saved":"images/capture_1.png","status":"ok"}
OK
Enviando captura 2...
{"saved":"images/capture_2.png","status":"ok"}
OK
Enviando captura 3...
{"saved":"images/capture_3.png","status":"ok"}
OK
Enviando captura 4...
{"saved":"images/capture_4.png","status":"ok"}
OK
Enviando captura 5...
{"saved":"images/capture_5.png","status":"ok"}
OK
Enviando captura 6...
```

*Correcto funcionamiento del servidor*



*carpeta con los resultados*



*ejemplo de resultado esperado*

## CONCLUSIONES

En esta práctica se ha aprendido a:

- Leer y procesar archivos JSON en C++.
- Validar y manipular matrices numéricas.
- Implementar interpolación bilineal.



## Practica 4 - SO

- Enviar datos mediante HTTP POST usando libcurl.
- Crear un servidor Flask para recibir datos.
- Visualizar matrices con Matplotlib.
- Generar automáticamente imágenes a partir de datos numéricos.
- Modularizar el código en C++ y Python.
- Resolver problemas de dependencias, entornos virtuales y librerías externas.

## INCIDENCIAS

Durante la práctica surgieron algunas incidencias como:

- Error al instalar paquetes Python por entorno gestionado (PEP 668).
- Necesidad de crear un entorno virtual para Flask.
- Caída del servidor por uso de Matplotlib en modo GUI.

Solucionado usando:

`matplotlib.use('Agg')`

- Errores de conexión cuando el servidor no estaba activo.
- Necesidad de ejecutar cliente y servidor en terminales separadas.
- Ajustes en libcurl para manejar errores de red.

```
(venv) marcos@MarcosDevice:~/sistemas_operativos/practica_4$ ./programa
Enviando captura 0...
Error en POST: Couldn't connect to server
ERROR
Enviando captura 1...
Error en POST: Couldn't connect to server
ERROR
Enviando captura 2...
Error en POST: Couldn't connect to server
ERROR
Enviando captura 3...
Error en POST: Couldn't connect to server
ERROR
Enviando captura 4...
Error en POST: Couldn't connect to server
ERROR
Enviando captura 5...
Error en POST: Couldn't connect to server
ERROR
Enviando captura 6...
Error en POST: Couldn't connect to server
ERROR
Enviando captura 7...
Error en POST: Couldn't connect to server
ERROR
```

*Error a la hora de conectar con el servidor*